QAntum senior software engineer take home exercise

Candidate brief

Design and build a small, useful first slice of a construction project team leader experience. Use coding agents throughout the exercise, and be ready to explain how you guided them, verified their work and made the key decisions.

About QAntum

QAntum is a passive fire compliance platform that captures the golden thread of building information, from design through installation to ongoing inspection. On-site operatives use an offline-first mobile app to install fire stopping against the solutions nominated for each site. They pin every penetration to a floor plan and record photo evidence of their work.

Back-office managers, quantity surveyors and clients can see those pins and progress live. Quality is checked as the work happens, and installations outside the specification are automatically flagged as variations.

The business problem

QAntum has a web app for administration, specification, and reporting, and an offline-first mobile app for operatives who survey work, install solutions and record evidence of compliance. There is no existing experience designed for team leaders.

Before going on site, a team leader needs to check that the materials required for the planned work are in stock and escalate shortages to others in the business. A shortage may mean choosing a different solution for the affected scenarios, or waiting until the required materials become available. Team leaders need this information to schedule their crews and work.

Choose a first slice that helps a team leader make a useful decision or take a useful action in this workflow. Explain how later iterations would extend it. You do not need to build the whole workflow.

Technology context

The current stack is React, Node.js, Flutter, Next.js, Supabase and Vercel. Take this stack, the existing web app and the offline-first mobile app into account in your design. Choose the components appropriate to your slice and explain how it could fit into QAntum. Justify any departures from the current stack.

Data supplied

You will receive solutions-excerpt.csv, an excerpt of 148 solution records for one supplier (Ryanfire). It includes internal and supplier reference codes, penetrating service, orientation, substrate, fire integrity and insulation requirements.

The file is a solutions catalogue. It includes required products, but not required material quantities, stock balances, delivery dates, nominated site solutions or crew schedules. Create clearly labelled sample data where your chosen slice needs it. Document your assumptions and any mappings you introduce.

If your slice includes alternative solutions, explain how suitability and approval would be established. A matching catalogue field alone should not be treated as proof that a substitution is 100% compliant.

Your task

- Understand the need and propose a short sequence of iterations that each add value. Select a first slice, identify its users and explain its value and boundaries.
- Use agents to help write a concise specification for that slice. Include functional requirements, acceptance criteria, relevant non-functional requirements, assumptions and exclusions.
- Create a technical design and architecture before building. Cover the user experience, data model, relevant components and interfaces, and how the solution fits the current stack. Explain the main trade-offs.
- Use coding agents to build a working solution for the selected slice. It should demonstrate the engineering care needed for something that could hypothetically be deployed in production. Identify any remaining gaps.
- Publish the code to a GitHub repository and deploy a publicly accessible front end experience, such as a web app at a public URL. Provide working CI/CD that runs checks and tests and deploys the solution to the cloud.
- Define a suitable test strategy and provide a complete, working set of tests for the slice. Cover the important behaviour, failure cases and risks, with automated tests running in CI.

Scope and judgement

Keep the implementation proportional to the problem. You choose whether the first slice needs persistence, authentication, offline behaviour, integrations or other capabilities. Make those decisions explicit and distinguish what works in the deployed exercise from what would need to change for production.

Use the supplied catalogue where it is relevant to your slice. Add any sample data required for your solution to be viable. Additional services and external integrations may be

represented with sample data or clearly identified substitutes. Explain what those choices let you demonstrate and what they leave untested.

What to submit

- A GitHub repository link and a public URL for the working experience, with any instructions needed to try it.
- A README explaining how to run the solution locally, run the tests and deploy it. Include a short demonstration scenario and identify sample data and known limitations.
- Concise Markdown files describing the iteration plan, first-slice specification, technical design and test strategy. Include the instructions and context files used to guide your coding agents. Organise these in whichever way makes the work easiest to review.
- Working application code, automated tests and CI/CD configuration. Make it possible to inspect the pipeline and see evidence of a successful deployment.
- A short account of your agentic coding approach: which tools you used, how you planned and directed the work, how you reviewed and verified outputs, and examples of where you corrected or rejected an agent's suggestion.

What we will assess

- Business understanding: does the solution address a real team leader need, and are the important uncertainties and assumptions visible?
- Slicing: does the first iteration deliver useful value, with clear boundaries and a credible path to subsequent iterations?
- Specification and agent guidance: are requirements, acceptance criteria and Markdown instructions specific enough to direct and verify the work?
- Technical judgement: are the architecture, technologies and functional and non-functional trade-offs appropriate to the requirements and size of the problem?
- User experience: can a team leader understand the information and take the intended action, including when information is missing or work is blocked?
- Engineering quality: does the deployed slice work, with a suitable test strategy, working tests and a functioning cloud delivery pipeline?
- Ownership: can you explain the implementation and its limitations, including the work produced by agents, to technical and less technical stakeholders?

Follow-up walkthrough

In a code walkthrough and panel interview, demonstrate your first slice and defend your agentic coding approach, technical design, user experience and trade-offs. Be ready to trace a user action through the implementation, explain your tests and deployment pipeline, and discuss what you would build next and why.

Keep it focused

We value a useful small slice and clear reasoning. We are not looking for long agent-generated narratives, coding without a specific architecture and technical design, or design and technology decisions you cannot justify. You remain responsible for the quality and correctness of the submission.